Neural Computing and Applications (2023) 35:10123–10145
https://doi.org/10.1007/s00521-023-08459-3

REVIEW

Data Augmentation techniques in time series domain: a survey and taxonomy

Guillermo Iglesias[1] · Edgar Talavera[1] · Ángel González-Prieto[2,3] · Alberto Mozo[1] · Sandra Gómez-Canaval[1]

Received: 6 August 2022 / Accepted: 3 March 2023 / Published online: 24 March 2023
© The Author(s) 2023

Abstract
With the latest advances in deep learning-based generative models, it has not taken long to take advantage of their remarkable performance in the area of time series. Deep neural networks used to work with time series heavily depend on the size and consistency of the datasets used in training. These features are not usually abundant in the real world, where they are usually limited and often have constraints that must be guaranteed. Therefore, an effective way to increase the amount of data is by using data augmentation techniques, either by adding noise or permutations and by generating new synthetic data. This work systematically reviews the current state of the art in the area to provide an overview of all available algorithms and proposes a taxonomy of the most relevant research. The efficiency of the different variants will be evaluated as a central part of the process, as well as the different metrics to evaluate the performance and the main problems concerning each model will be analysed. The ultimate aim of this study is to provide a summary of the evolution and performance of areas that produce better results to guide future researchers in this field.

Keywords Deep learning · Time series · Data augmentation · Generative models

1 Introduction

From the advent of deep learning (DL), a large part of research and the efforts of the scientific community have focused on solving and improving supervised training tasks. Supervised learning requires larger datasets, with a large number of different features, and these samples also need to be labelled in order to be feasible. With the latest models, it is becoming more difficult to obtain a suitable dataset, as the models require larger amounts of data to carry out the training. Usually, there exists a large number of public repositories from which to obtain suitable datasets for training in most application areas.

However, in the time series domain, datasets are not that easy to access, where there are usually a number of privacy issues, and it is typically difficult to obtain large enough or balanced datasets. This often leads to a major problem when attempting to train one of these models on an incomplete, unbalanced or privacy-challenged dataset. Typically, these problems are addressed by pre-processing dataset techniques, such as subsampling, or in datasets that are not large enough, by data augmentation (DA) techniques [1, 2].

Nevertheless, as soon as problems arise, technology evolves to address these boundaries. In recent years, artificial neural network (ANN) and their application to the field of DL have experienced a period of great advances. Although a multitude of models has contributed to this expansion, one of the most revolutionary that has been

✉ Edgar Talavera
e.talavera@upm.es

Guillermo Iglesias
guillermo.iglesias@upm.es

Ángel González-Prieto
angelgonzalezprieto@ucm.es

Alberto Mozo
a.mozo@upm.es

Sandra Gómez-Canaval
sm.gomez@upm.es

[1] Departamento de Sistemas Informáticos, Universidad Politécnica de Madrid, Madrid, Spain

[2] Departamento de Álgebra, Geometría y Topología, Universidad Complutense de Madrid, Madrid, Spain

[3] Instituto de Ciencias Matemáticas (CSIC-UAM-UCM-UC3M), Madrid, Spain

proposed appeared in 2014 by Ian Goodfellow [3], with his generative adversarial network (GAN).

GAN is certainly not the earliest generative architecture ever introduced; already in 1987 Yann Lecun [4] suggested in his thesis the autoencoder (AE) architectures, which were capable of generating data modifications received as input. But it is not until the incorporation of directed probabilistic models into AE architectures, also known as variational autoencoder (VAE) [5], that the models begin to be presented as capable of generating synthesised data.

Although these networks show impressive results, the capabilities of GAN have been shown to be far ahead, with impressive results applied to the field of imaging. Furthermore, this is not the unique area of application. Synthetic data generation is a powerful boost for synthesis of sensitive data, such as in the world of telecommunications.

There are many time series applications where these algorithms have shown good results when improving the capabilities of the models by enlarging the datasets [6, 7]. In this area, time series data are especially sensitive due to aspects such as the low availability of high-quality datasets or privacy of the data [8–11].

All these advances in the DL area allow us to synthetically increase the size of the datasets used in machine learning (ML) tasks. As mentioned above, the size of the dataset is a sensible characteristic that can affect the performance of the developed models.

Regarding time series data, the availability of these huge datasets is even more complicated. There are other fields such as image processing or natural language processing (NLP) where data is much more available. But in the time series domain, it is more difficult to obtain these samples [9–11]. In this sense, DA arises as a technique to counter the scarcity of data.

Another important factor is that time series data have particularities when it comes to processing them. Each dataset of time series is very different and needs special attention on how it is being augmented. Therefore, the proposed techniques for time series data must be analysed and discussed to better understand which technique can be used in which data.

Thus, this paper aims to review all the existing technologies for DA, and to review the positive and negative aspects of each of them. This review can help researchers better understand how DA techniques can be applied to time series data to obtain better results when training machine learning (ML) models. We also expect that it will be useful to highlight the main differences between time series data and other domains.

1.1 Paper structure

The rest of the paper is structured as follows: Sect. 2: "Problem Statement" presents the review in its context, providing a view of the importance of this work in the current context; Sect. 3: "Related Works" reviews previous similar works in this area, highlighting the differences between the previous ones and the present manuscript; Sect. 4: "Background" introduces a technical background of the performance of the techniques used to augment time series data; Sect. 5: "Evaluationmetrics" explains the problematic associated with how to evaluate the results of the new synthetic data; Sect. 6: "Data Augmentation algorithms review" presents the most important and current work in this area, reviewing the technical aspects of each approximation; Sect. 7: "Discussion" discusses the results and behaviour of each algorithm presented in the previous section and finally Sect. 8: "Conclusion" summarises the main conclusions of the research.

2 Problem statement

This study addresses the current context of generating new data samples in temporal series datasets. The purpose of this paper is to focus on the most relevant and cutting-edge research in this area to provide an accurate and complete view of this field. This research tries to cover all the possible techniques used to enhance this type of data.

In contrast to previous works, the aim of this survey is to provide a complete view of how different approaches to this problem have been developed. One of the main problems of reviewing a portion of the total techniques used in this area is that it does not fully explain how the current paradigm is structured. This is particularly critic when comparing different techniques, where it is desirable to have all the approximations in the area to fully understand the particularities of each one.

In this sense, it is important to provide an updated and comprehensive study of the state of the art in this area. This manuscript is focused on the three most important pillars of current data augmentation in time series: traditional algorithms VAE and GAN.

3 Related works

In the latest times, several high-quality data augmentation review papers have been published [12–14]. However, most of them are focused on more popular areas such as imaging, video or natural language processing (NLP). Although these techniques focus on correcting the

imbalance or incompleteness of the dataset, there are other areas of application where these problems are more common. The scarcity of valid datasets is not as clear in all areas of DL applications as in time series.

In a first approach to the literature review, in [15] an approximation of DA algorithms is made for use in neural network algorithms for time series classification. In the survey, they evaluated 12 methods to enhance time series data in 128 time series classification datasets with six different types of neural networks. Other recent studies focus more specifically on the use of GANs for data augmentation, as in [16], where they analyse the taxonomy of discrete-variant GANs and continuous-variant GANs, in which GANs deal with discrete time series and continuous time series data. These surveys analyse DA in time series using neural networks, but lack a comparison of these algorithms with more traditional approaches.

However, improving data sources to feed artificial intelligence (AI) algorithms is not limited to DA exclusively. Therefore, some studies have decided to take the path of building synthetic traffic generators to build their datasets almost from scratch; some examples focusing on this aspect are [17–19]. In this way, they are able to abstract from the dataset itself, which is only necessary to understand the distribution of the data. Furthermore, in [20] they set out a further study of the repercussions of these technologies, highlighting one of the major advantages of generating synthesised data, the abstraction of privacy issues and the ease of obtaining datasets.

Despite the possibilities presented by new technologies to improve the quality of time series datasets, there are no studies that compile all technologies, with a comprehensive comparison of them. Furthermore, to the authors' knowledge, a survey that compiles all the different techniques is missing in the literature. This work's goal is precisely to address this problem. It is crucial to develop a review that studies and compares all novel techniques proposed in time series domain. Previous works were centred on a specific model or approximation, with works such as [16, 21, 22]. The objective of this survey is to have a wider view of the field, and to be able to further compare each technology and different approximations. With respect to previous work, a complete review is presented, extending the explanation of each algorithm's performance, results, advantages and disadvantages.

In addition, this field is in constant evolution, so it is common for reviews to become outdated due to the publication of new articles. It is considered very important to provide an updated study of the state of the art that follows the latest trends in this area.

Therefore, the goal of this review is to contribute to reducing the existing gap in the area by trying to bring together all the time series DA algorithms that currently exist, contrasting their possible virtues, approaches and differences to help future researchers position themselves in the area.

4 Background

4.1 Traditional algorithms

DA has been a crucial task when the available data are unbalanced or insufficient. Traditionally, in fields such as image recognition, different transformations have been applied to data such as cropping [23–26], scaling [24, 25], mirroring [23, 26, 27], colour augmentation [23, 25, 28] or translation [27].

These algorithms cannot be applied directly to time series given the particularity of the time series data distribution [29]. For example, if one wants to apply rotation to augment an image dataset, it is possible to rotate each image to generate new ones. This cannot be directly done in the time series domain, e.g., if a time series sample is divided into several portions and these portions are reorganised using linear interpolation between them, the result would not be valid because the tendency of the data would be destroyed. Due to the diversity of the time series data, not all techniques can be applied to every dataset. Some of the previous algorithms used in computer vision must adapted to a time series domain, but, in other cases, new specific algorithms must be designed to treat with time series data.

Another important factor when applying DA to the time series domain, especially in signal processing, is that manipulation of the data could distort the signal too much, leading to negative training.

Traditional algorithms are defined as all the techniques based on taking data input samples and synthesising new samples by modifying these data and applying different transformations. The main difference between this technique and those that are reviewed in Sects. 4.2 and 4.3 is that, in the former algorithms, the transformations are applied directly to the data, while in the latter the objective is to learn the probability distribution of the data in order to generate completely new samples trying to imitate the data distribution.

4.2 Variational Autoencoder (VAE)

VAEs are neural generative models first introduced by Diederik P. Kingma and Max Welling [5]. This algorithm is based on the AE architecture [4] proposed in 1987. AEs allow changing typical artificial intelligence problems, such as linear regression or classification, to domain-shifting problems. In order to perform this, AEs take an

input, usually an image, and infer as the output modifications of that same input. This is known as self-supervised training, where the objective is to obtain the input with slight modifications as an output. One of the most popular applications of this model is image denoising [30]. In this case, the input is an image that contains noise and the output should be the input image without the undesired noise.

AE Network is composed of two components, an encoder and a decoder. The encoder is in charge of reducing the input dimensionality of the data to a latent space, while the decoder reconstructs the input information from this latent representation. This latent space is a lower-dimensional manifold of the input data. Then, synthetic data are generated, interpolating the values of the latent space and decoding them. However, this interpolation of the latent space does not generate completely new values; it just mixes the features of the learned probability distribution.

In order to avoid the overfitting produced in AE, VAE regularises its training, generating more diverse samples. The main difference between both architectures is that VAE encodes the input information in a probability distribution rather than in a point. Then, from this distribution, it samples a point that is then decoded to synthesise new samples.

This intermediate step allows the network to map the input distribution to a lower-dimensional distribution from which new latent points can be generated. To do so, the latent distribution is normally defined by a normal distribution with a mean $\overrightarrow{\mu} = (\mu_1, \ldots, \mu_n)$ and a standard deviation $\overrightarrow{\sigma} = (\sigma_1, \ldots, \sigma_n)$. These mean and standard deviation vectors define the latent distribution of the model.

Leaving the network to learn a distribution, instead of a set of points learned in AE, the decoder network associates the features of the input data with the probability areas with their respective mean and deviation. With this representation, the mean of the distribution defines the centre point from which the synthetic samples will be generated and the standard deviation defines the variability in the output, that is, the diversity of the generated samples.

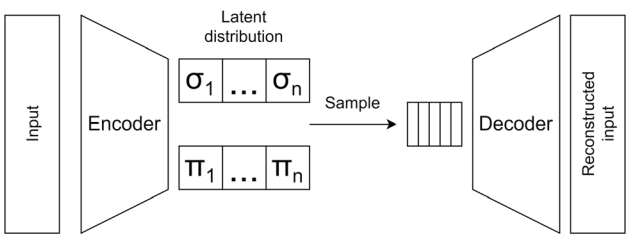


Fig. 1 VAE architecture with the both latent space representations (π and σ)

Figure 1 shows the architecture of a VAE network.

Regarding the training of a VAE network, there are two different loss functions. The *reconstruction term* is in charge of the reconstruction of the input data. It measures the error of the network when it builds. This metric acts the same as the error of a standard AE.

On the other hand, VAE include a *regularisation term* that tries to organise how the latent distribution generates new latent spaces. The function of this term is to measure the distance of the sampled data points and a Gaussian distribution. The distance used to measure this error is the Kullback–Leibler divergence (KL divergence) [31]. So in order to calculate the loss of the VAE both errors are added, measuring at the same time the reconstruction error of the output and the error of sampling points following a normal distribution.

4.3 Generative Adversarial Network (GAN)

GAN is a generative neural model based on a competition between two neural network (NN). They were first introduced by Ian Goodfellow [3] in 2014. The objective of the architecture is to replicate a given data distribution in order to synthesise new samples of the distribution. To achieve this goal, the GAN architecture is composed of a generator (G) model and a discriminator (D) model. The former is in charge of generating the synthetic samples of the data distribution, while the latter tries to distinguish the real samples from the synthesised samples.

To accomplish such generation of completely new data that are indistinguishable from the input data distribution, both models interact with each other. The model generator (G) generates samples trying to replicate, without copying, the distribution, while the model discriminator (D) discriminates the real samples from the fake samples. In this way, when discriminator (D) differentiates both distributions, it feedbacks generator (G) negatively; on the other hand, when discriminator (D) is not capable of differentiating each distribution, its positively feedbacks generator (G). In doing so, generator (G) evolves to fool discriminator (D). At the same time, discriminator (D) is positively rewarded when discrimination is done correctly.

This competition encourages both networks to evolve together. If discriminator (D) fails in its task, generator (G) will not evolve because it will always succeed, despite the quality of the synthesised samples. Even if discriminator (D) always perfectly distinguishes both distributions, generator (G) will not be able to fool discriminator (D), making it impossible to evolve. The standard GAN architecture is depicted in Fig. 2.

From a mathematical perspective, this competitive behaviour is based on Game Theory, where two players compete in a zero-sum game. The discriminator

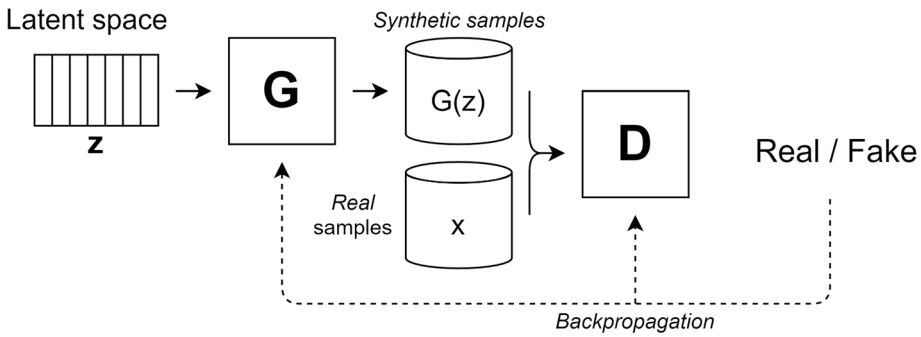


Fig. 2 GAN architecture

(D) estimates the *a posteriori* probability $p(y|x)$, where y is the label (true or fake) of the given sample x. And generator (G) generates synthetic samples from a latent vector z, which can be denoted as $G(z)$.

From a formal point of view, this competition is defined as a minimax game where discriminator (D) tries to maximise its accuracy when discriminating between both distributions and generator (G) tries to minimise this accuracy. The formulation of this process is denoted as follows:

$$\min_G \max_D L(D,G) = \min_G \max_D E_{x\sim p_r} \log[D(x)] + E_{z\sim p_z} \log[1 - D(G(x))], \quad (1)$$

where z is the latent vector, which is generated randomly by a uniform or Gaussian distribution p_z and $x \sim p_r$ is the real distribution.

In the publication in which GAN was presented [3], it was proved that the architecture can converge to a unique solution. This point, known as Nash equilibrium, is characterised by the fact that none of the networks can reduce their respective losses. This optimal result is very difficult to achieve in reality due to the instability behaviour of GAN. The Nash equilibrium is, in fact, most of the time not achieved because of the constant competition between both networks.

Respecting DA, GANs synthesise completely new samples, using the dataset distribution as their base on learning the underlying data distribution. Therefore, GANs are able to produce more diverse samples respect with respect to previous approximations. This is known as data generation, where the main difference between data generation and DA is that the former generates new samples of synthetic data while the latter use the original samples to produce new ones using their information.

5 Evaluation metrics

Due to the particularities of the time series field, there is no unique metric to evaluate the reliability of algorithms in all of their applications. Finding a measurement capable of evaluating the quality and diversity of the synthesised data is still an open issue.

For example, in GAN networks, there exists no consensus between the different studies about the evaluation metrics to use. In addition, most of the evaluation metrics designed are centred on computer vision, since it is the most popular field for this kind of network.

Therefore, it will be described the most commonly used metrics that have been used to evaluate the algorithms that will be discussed in this article. However, it should be noted that to choose a proper evaluation metric, one should adapt the metric to the specific data augmentation algorithm and the application field.

5.1 External performance evaluation

When applying DA to a dataset, the most common objective is to generate new data samples to improve the performance of certain models, reducing the imbalance of the data or the lack of data. One of the most popular ways to measure how the addition of new data changes the behaviour of the models is simply to compare these models before and after DA. Then, it is possible to compare whether each model has improved its performance after applying DA to the input data.

This approximation is purely practical and relies on the correlation between the performance of a defined model and not the quality of the synthetic samples themselves. Most traditional algorithms base their performance on this method because it is a straightforward method to evaluate an algorithm.

In [32], the performance of the DA algorithms they propose is achieved using *symmetric Mean Absolute Percentage Error* and *Mean Absolute Scaled Error*, which are the two most common evaluation metrics used in forecasting. This research compares the values of these metrics before and after applying DA to the dataset, then evaluates how the models improve their performance due to the addition of more data to the training set.

In [15], they used six different neural networks to evaluate how each DA algorithm affects the classification

of the data. In particular, they evaluated VGG [24], residual network [25], multilayer perceptron [33], long short-term memory [34], bidirectional long short-term memory [35] and long short-term memory fully convolutional network [36]. Then, the changes in the accuracy of the models are compared, observing how certain DA algorithms benefit the performance of the models, while in other cases it gets worse. The main drawback stated in the article is that each architecture has its particularities, given the different results for each algorithm and making it a difficult task to differentiate the best algorithm. Furthermore, because all of them are neural models, it is difficult to interpret some of the results.

The approach followed in [29] is to compare different DA techniques by the increase in accuracy produced in each case of study. It is worth mentioning how this approximation adapts to each application without having to change anything. The authors of the article are able to compare very different techniques such as noise addition, GAN, sliding window, Fourier transform and recombination of segmentation under the same criteria for a specific domain purpose. This example shows how this approach easily adapts to different DA techniques, making it possible to compare the results for a certain task.

A similar strategy to measure the quality of the generated data is to compare different models using a defined loss function. This approach was followed in GAN architectures in works like [3, 37–40], where a comparison between networks is possible using the same loss function to evaluate their training. Then, they correlate the quality of the synthetic data with this value. This strategy can be applied naturally to the time series domain, allowing comparison between different networks. However, the main drawback of this method is that it compares the performance of different neural models and cannot be applied to other models. It should be noted that, as in previous metrics, it correlates the quality of the generated data not with the data itself but rather with the performance of the model.

In [41], they compare the performance of different DA techniques with mean per-class error (MPCE). This metric, proposed in [42], measures the error per class in J datasets taking into account the number of classes in each dataset. The main particularity of mean per-class error (MPCE) is that it allows us to quantify the performance of an algorithm for different datasets. The mean per-class error (MPCE) is calculated as follows:

$$\text{MPCE} = \sum_{j \in [J]} \text{PCE}_j = \frac{e_j}{c_j} \tag{2}$$

where e_j is the error rate and c_j is the number of classes in each dataset. This metric is capable of taking into account the number of classes of each dataset, in order to normalise the comparison between different sources of data.

5.2 GAN related metrics

Since the introduction of GAN, it has always been an open issue to measure the quality of the synthesised samples produced by the architecture [43]. One of the most important difficulties when designing a metric for GAN is the ability to capture both the quality and diversity of the generated data.

In addition to being still an open issue, there is consensus on some metrics and many papers measure their results with the same metrics [44–49]. The main problem in the time series domain is that it is not always possible to adapt the metrics to the particularities of this field because most of the metrics are designed to be useful in computer vision-related tasks.

Over the past few years, some works have suggested applying DA to time series data and treating it as if it were an image [50, 51]. These papers use GAN networks to synthesise new time series data, but to convert the signal data into an image. In these cases, traditional GAN metrics, such as the Inception Score [52], Mode Score [53] or Fréchet Inception Distance [54] are used to evaluate the results. These metrics are based on how the *Inception v3* neural classifier distinguishes the different samples. The idea is to measure the entropy of the synthetic dataset using an external classifier.

In addition to the field of computer vision, studies have been developed that apply GAN directly to time series. That is, in TimeGAN [55] two new metrics are proposed to assess the quality of the generated samples. The Discriminative Score is based on the use of an external pre-trained model, as was done with the Inception Score, consisting of a 2-layer LSTM. The Discriminative Score measures how this model distinguishes between real and fake samples and the classification error corresponds to the Discriminative Score. The Predictive Score measurement was introduced in [56] with the name Train Synthetic, Test Real (TSTR) which also uses a 2-layer LSTM, but in this case, this model is trained with synthetic samples. The model is then evaluated using the original dataset. The Predictive Score corresponds to mean absolute error (MEA) of the model trained with the synthetic samples evaluated with the real samples. This metric is, at the moment, one of the most effective and used evaluation metrics.

5.3 Similarity measurements

This set of metrics focuses on the comparison of two probability distributions. The idea is to measure how far from the original distribution the synthetic samples

generated with DA. The main advantage of these metrics is that they focus on directly studying the quality of the data, in contrast the previously reviewed methods that measured the quality indirectly. Another advantage of these types of metrics is that they can be applied to synthetic data despite the algorithm used to generate them.

An empirical and qualitative approach to measuring the differences between two distributions is to reduce the dimensionality of the data and perform a visual comparison. The objective is to reduce the dimensionality of the data to plot the samples in a bidimensional space; an empirical comparison is then made by visualising the data. This approach was followed in [57] where they applied *t*-distributed stochastic neighbour embedding (t-SNE) and principal component analysis (PCA). Then, they compared the distribution of the data in the two-dimensional space for TimeGAN [55], recurrent conditional GAN (RCGAN) [56], continuous recurrent GAN (C-RNN-GAN) [58], T-Forcing [59], WaveNet [60] and WaveGAN [61]. This approach was also followed in [15] where they used principal component analysis (PCA) to compare different traditional algorithms for the GunPoint dataset from the 2018 UCR Time Series Archive [62].

Kullback–Leibler divergence (KL divergence) has been used in work such as [63, 64] to measure similarities between synthetic and real datasets. Recall that Kullback–Leibler divergence (KL divergence) [31] is defined as

$$D_{KL}(P \,||\, Q) = \sum_i P(x_i) \log\left(\frac{P(x_i)}{Q(x_i)}\right), \tag{3}$$

where P and Q are the probability distributions whose distance is calculated and i runs over the samples x_i of the distribution. This Kullback–Leibler divergence (KL divergence) [31] is not a symmetric distance, so it can be symmetrized to give rise to the so-called Jensen–Shannon divergence (JSD), defined as

$$JSD(P \,||\, Q) = D_{KL}(P \,||\, (P+Q)/2) + D_{KL}(Q \,||\, (P+Q)/2).$$

In [65], a novel measurement is proposed to quantify the distance between the time series distribution. It is based on calculating the Wasserstein distance between time series data. The metric is defined by measuring the Wasserstein distance of the energy between frequencies. The Wasserstein–Fourier distance between the probability distributions is computed as follows:

$$\mathrm{WF}([x],[y]) = W_2(s_x, s_y) \tag{4}$$

where s_x and s_y are the normalised power spectral densities of the distributions.

6 Data Augmentation algorithms review

During this section, different state-of-the-art algorithms will be reviewed. This section will explain the particularities, strengths and weaknesses of each algorithm. In addition, the different approximations to apply DA will be grouped and related between them. A taxonomy of the different trends and lines of research will be proposed, showing the different existing links between the works of the last years.

It should be noted that not all the algorithms can be applied to all types of time series data; in some cases, the algorithms proposed will be heavily focused on a certain application, while in others more general techniques will be studied.

6.1 Basic DA Methods

The basic DA algorithms that will be reviewed in this section are all techniques that use data manipulation to generate new synthetic data samples using existing samples and transform the original samples. All these techniques have as their base the deformation, shortening, enlargement or modification of the data samples of the dataset. This group of techniques has been traditionally used in fields such as computer vision and, in some cases, the same algorithms can be adapted to process time series data, but in others, new algorithms must be designed specifically to use time series data as input.

Therefore, the most important traditional algorithms that have been applied to time series data will be reviewed and discussed, outlining their particularities, advantages and disadvantages. Figure 3 shows the taxonomy proposed for the different algorithms reviewed.

6.1.1 Time slicing window

Slicing, in time series, consists of cutting a portion of each data sample, to generate a different new sample. Normally, slicing is applied to the last steps of the sample, but the snippet of the original sample can be obtained from any step. When the original data is cropped, a different sample is produced, but unlike image processing, it is difficult to maintain all the features of the original data. The process of slicing time series data provides new data given as:

$$x'(W) = \{x_\varphi, \ldots, x_t, \ldots, x_{\varphi+W}\}, \tag{5}$$

where W is the slice window that defines the crop size and φ is the initial point from where the slicing is performed, such as $1 \leq \varphi \leq T - W$. One of the most important drawbacks of slicing the signal is that it can lead to invalid

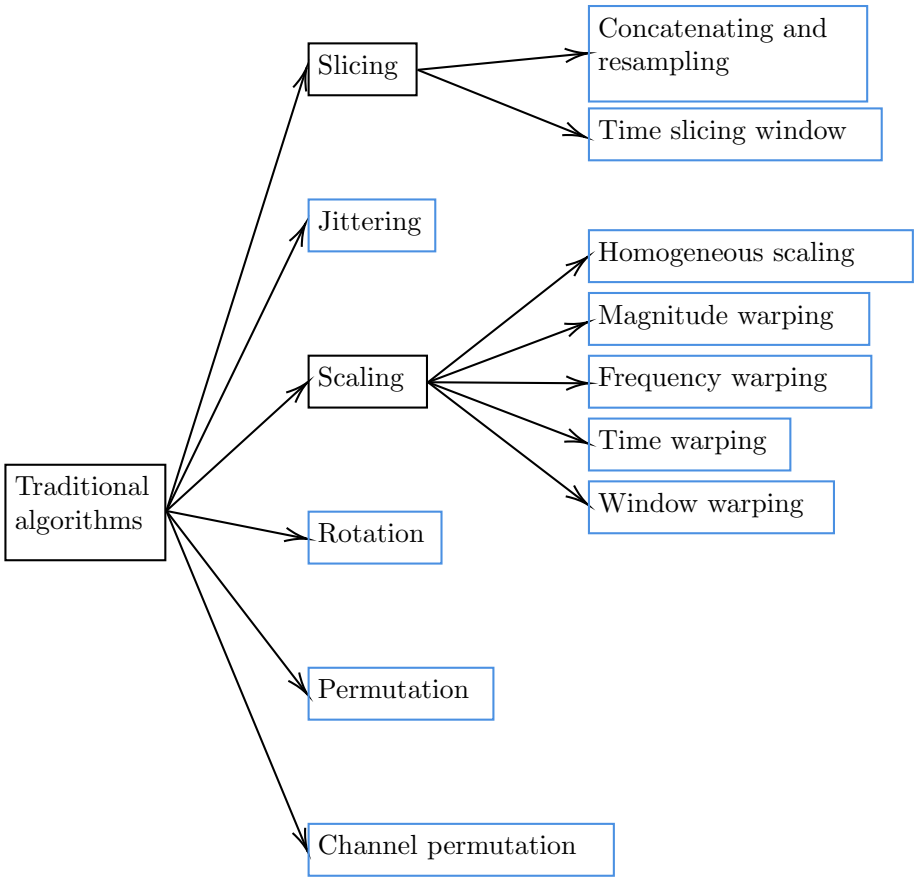


Fig. 3 Traditional DA algorithms taxonomy

synthetic samples because it can cut off important features of the data.

A variation of the slicing method is proposed in [66], where the concatenating and resampling method is presented. This algorithm first detects features in the data, called characteristic points. This is made by using the Pan-Tompkins QRS detector [67]. This algorithm detects the characteristic points in a heartbeat signal, so in order to apply the concatenating and resampling algorithm it must be defined and algorithm to detect these points. Then, after detecting the characteristic points, it is defined a subsequence that starts and ends in a characteristic point. This sequence is replicated several times and sliced in a window to perform DA.

This variation was applied to electrocardiogram (ECG) data of variable length between 9 and 61 s sampled at 300 Hz.

The concatenating and resampling algorithm tries to ensure the validity of the data, taking into account that the signal maintains its features. But the main disadvantage of this method is that it needs a detector of characteristic points that ensure the data validity.

6.1.2 Jittering

Jittering consists of adding noise to time series to perform DA. This technique, in addition to being one of the simplest forms of DA, is one of the most popular in time series [68, 69]. Jittering assumes that the data are noisy which, in many cases, i.e., when dealing with sensor data, is true.

Jittering tries to take advantage of the noise of the data and simulate it to generate new samples. Typically, Gaussian noise is added to each time step; the mean and standard deviation of this noise define the magnitude and shape of the deformation, so it is different in each application. The jittering process can be defined as follows:

$$x'(\epsilon) = \{x_1 + \epsilon_1, \ldots, x_t + \epsilon_t, \ldots, x_T + \epsilon_T\}, \quad (6)$$

where ϵ stands for the noise addition at each step of the signal.

As mentioned above, the jittering process must be adapted to each case, because there are cases such as [70] where the effects of jittering lead to negative learning. In this research, it was used as time series data the information received by a wearable sensor, capturing 58 s at 62.5 Hz that were later resampled to 120 Hz per sample.

6.1.3 Scaling

Scaling consists of changing the magnitude of a certain step in the time series domain. The idea is to maintain the overall shape of the signal while changing its values. With scaling, the new generated data change the range of values, but keep the shape of the changes. Homogeneous scaling is given as:

$$x'(\alpha) = \{\alpha x_1, \ldots, \alpha x_t, \ldots, \alpha x_T\}, \tag{7}$$

where $\alpha > 0$ defines the scale of the change. This value can be defined by a Gaussian distribution with mean 1 and with σ as a hyperparameter [70], or it can be previously defined from a list of values [69].

Within scaling techniques, there are several different approximations for a specific time series domain. They take advantage of the specific properties of the signal data and adapt to perform DA.

Magnitude warping is a technique used in [70] that consists of an application of a variable scaling to different points of the data curve. To define where to apply the transformation, a set of knots $\mathbf{u} = u_1, \ldots, u_i$ is defined; these represent the step in which the scaling is performed and their values are generated by using a normal distribution. Then, the magnitude of the scaling is defined by a cubic spline interpolation of the knots $S(\mathbf{x})$. Then, the magnitude warping can be defined as follows:

$$x'(\alpha) = \{\alpha_1 x_1, \ldots, \alpha_t x_t, \ldots, \alpha_T x_T\}, \tag{8}$$

where $\alpha = \alpha_1, \ldots, \alpha_i = S(\mathbf{x})$. With magnitude warping, the main particularity is that it applies a smoothened scaling to each point of the curve, multiplying the possibilities of the transformation while preserving the overall shape of the data. However, it still assumes that the synthetic data maintain validity after transformation.

This technique was used in [70] where the data was captured from a wearable device to detect if a patient suffers from Parkinson's disease.

Frequency warping is a variation of magnitude warping, mostly applied in speech processing [71–73]. The most popular version in speech recognition is vocal tract length perturbation, which can be applied in a deterministic way [72] or stochastically within a range [74]. In particular, this technique was used in [72] where a dataset of human conversation sampled at 8 KHz was used.

Another scaling technique is time warping, the idea is very similar to magnitude warping, but the main difference between both algorithms is that time warping modifies the curve in the temporal dimension. That is, instead of fluctuating the magnitude of the signal in each step, it stretches and shortens the time slices of the signal. To define how to warp the signal, a smooth curve, as was done in magnitude warping, is defined by using a cubic spline for a set of knots. The time-warping algorithm can be denoted as:

$$x'(\tau) = \left\{x_{\tau(1)}, \ldots, x_{\tau(t)}, \ldots, x_{\tau(T)}\right\}, \tag{9}$$

where τ defines the magnitude of the warp, this function is generated by using a cubic spline $S(u)$ between different knots generated using a normal distribution. This algorithm has been used in several works, such as [75, 76]. There is yet another variation of this algorithm, known as window warping, followed in [77] that defines a slice in the time series data and speeds up or down the data by a factor of 1/2 or 2. In this case, the warping is applied to a defined slice of the whole sequence; the rest of the signal is not changed.

6.1.4 Rotation

Rotation can be applied to multivariate time series data by applying a rotation matrix with a defined angle. In univariate time series, rotation can be applied by flipping the data. Rotation is defined as follows:

$$x'(R) = \{Rx_1, \ldots, Rx_t, \ldots, Rx_T\}, \tag{10}$$

where R is the rotation matrix used to twist the data. This algorithm is not very usual in time series due to the fact that rotating a time series sample could make it lose the class information, as it happened in [78], where it was used the UCR archive [62] with various dataset from different real-world applications were the composition of the samples of each dataset varies. On the other hand, there have been articles [70] that demonstrate the benefits of applying rotation, especially combined with other data transformations, in this case, using wearable data samples at 120 Hz.

6.1.5 Permutation

Shuffling different time slices of data in order to perform DA is a method that generates new data patterns. It was proposed in [70], where a fixed slice window was defined from which the data is rearranged, but it has also been applied with variable windows, as it was done in [79] using electrocardiogram (ECG) data at 300 Hz. The main problem of applying permutation is that it does not preserve time dependencies; thus, it can lead to invalid samples. The permutation algorithm can be denoted as follows:

$$x'(w) = \left\{x_i, \ldots, x_{i+w}, \ldots, x_j, \ldots, x_{j+w}, \ldots, x_k, \ldots, x_{k+w}\right\} \tag{11}$$

where i, j, k represents the first index slice of each window, so that each is selected exactly once, and w denotes the window size if the slices are uniform $w = T/n$ where n is the number of total slices.

6.1.6 Channel permutation

Changing the position of different channels in multidimensional data is a common practice. In computer vision, it is quite popular to swap the RGB channels to perform DA [80]. With respect to time series, channel permutation can be applied as long as each channel of the data is still valid. The channel permutation algorithm, for multidimensional data such as $x = \{\{x_{11}, \cdots, x_{1T}\}, \cdots, \{x_{c1}, \cdots, x_{cT}\}\}$ where c is the number of channels, is given by

$$x = \left\{\left\{x_{\sigma(1)1}, \ldots, x_{\sigma(1)T}\right\}, \cdots, \left\{x_{\sigma(c)1}, \ldots, x_{\sigma(c)T}\right\}\right\}, \quad (12)$$

where $\sigma : \{1, \ldots, c\} \rightarrow \{1, \ldots, c\}$ is the used permutation of the channels.

In the time series domain, this algorithm is not applicable to the application of the data, because permutation assumes that the channel information is independent of the channel itself. In other words, the information about the channels is not linked to the particular channel.

That is, in [63], they applied this algorithm by flipping the position of the sensors that recorded the data signals, recording the data at 20 Hz using a window of 6 s. In the article, the researchers used an exercise mat with eight proximity sensors that they flipped to generate new data. That is, in practice, changing the position of the signal channels.

6.1.7 Summary of the traditional algorithms

Figure 4 shows an example of each algorithm reviewed:

6.2 Data augmentation through VAE

The use of AE architectures is nothing more than the evolution of data generation algorithms to produce more and better data, which means that, better, they are varied and therefore the standard deviation with respect to the original data is perfect. To precisely control the deviation of the data, VAE arises as the evolution of AE to generate better synthetic data, as shown in [81] where VAE is used to generate data for anomaly detection problems with LSTM. Or this other work [82], in which they use a dataset augmented with VAE to improve the recognition of human activity with LSTM. Even more exhaustive studies [83, 84] show the efficiency of these algorithms in increasing the size of datasets.

But the use of VAEs for DA is not only intended for neural network models, but can also improve results when traditional machine learning algorithms are applied [85]. However, they can also be used in applications with unsupervised training, that is, in [86], which applies them to unsupervised domain adaptation for robust speech recognition.

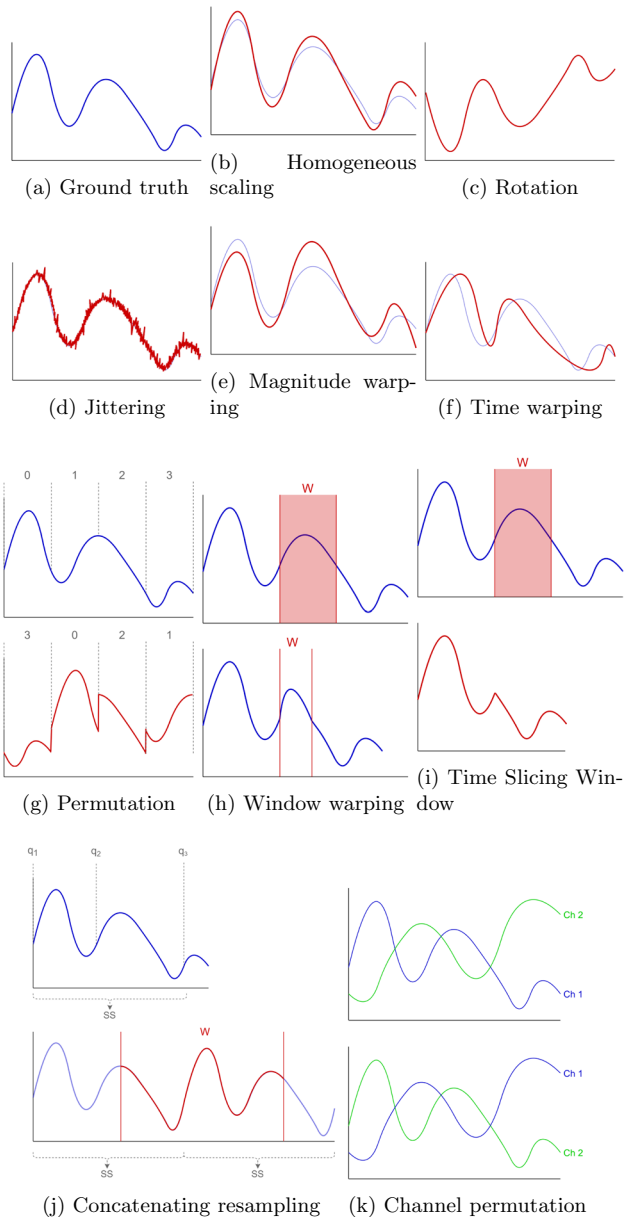


Fig. 4 Summary of traditional algorithms

In [87], they point out that most data augmentation methods for time series use feature space transformations to artificially enlarge the training set; they propose a composition of autoencoders (AEs), variational autoencoders (VAEs) and Wasserstein generative adversarial networks with gradient penalty (WGAN-GPs) for time series augmentation.

In the end, each VAE model and its hyperparameter configuration make them specialise in the area or format of the dataset they want to work on; but above all to the type of problem for which it will be used afterwards. That is to say, what makes the difference between the models is what the generated data will be used for, regarding problems

such as: classification, forecasting, value imputation or prediction.

6.2.1 Taxonomy for the VAE algorithms reviewed

Figure 5 shows a scheme to group the different investigations reviewed in Sect. 6.2, this way all the VAE algorithms for DA can be viewed schematically.

6.2.2 VAE for anomaly detection

As mentioned before, VAE are a DA architecture that has been widely used in the field of anomaly detection. The main objective of using these models in anomaly detection tasks is to be able to generate data in order to avoid the lack of invalid data of the datasets. The most common scenario is that there are not enough available anomalous samples in order to train machine learning models with the dataset, so the use of VAE is focused on generating new data.

The work presented in [81] is centred on the classification of electrocardiogram (ECG) signals, distinguishing between the ones with cardiac dysfunction. The data used consists of windows of 3600 samples downsampled to 905 sample points during a period of time of 10 s. In order to augment the data available it is used a conditional VAE (CVAE) [91] that is able to learn which samples are normal and which are anomalous. This conditional VAE (CVAE) architecture is composed of LSTM layers [92] which process the temporal data of the electrocardiogram (ECG) signals.

Another architecture based on the anomaly detection problem is the smoothness-inducing sequential VAE (SISVAE) [88] which uses a VAE with recurrent layers to maintain temporal dependencies. This work focuses on the problem of abrupt changes between time steps, which led to non-smooth reconstructions of the input data of the model, and therefore temporal abrupt changes in the synthesised samples. The mechanism to avoid this is to introduce a corrective bias for each time step of the signal, calculated using the Kullback–Leibler divergence (KL divergence) [31] between one point and the next one in the series. The results of the work are tested using two different time series synthetic datasets.

6.2.3 VAE for data imputation

One field where the VAE architecture has been widely used is in data imputation tasks. This process consists of generating new data in a sample where there is missing information. In the temporal series domain, this process is

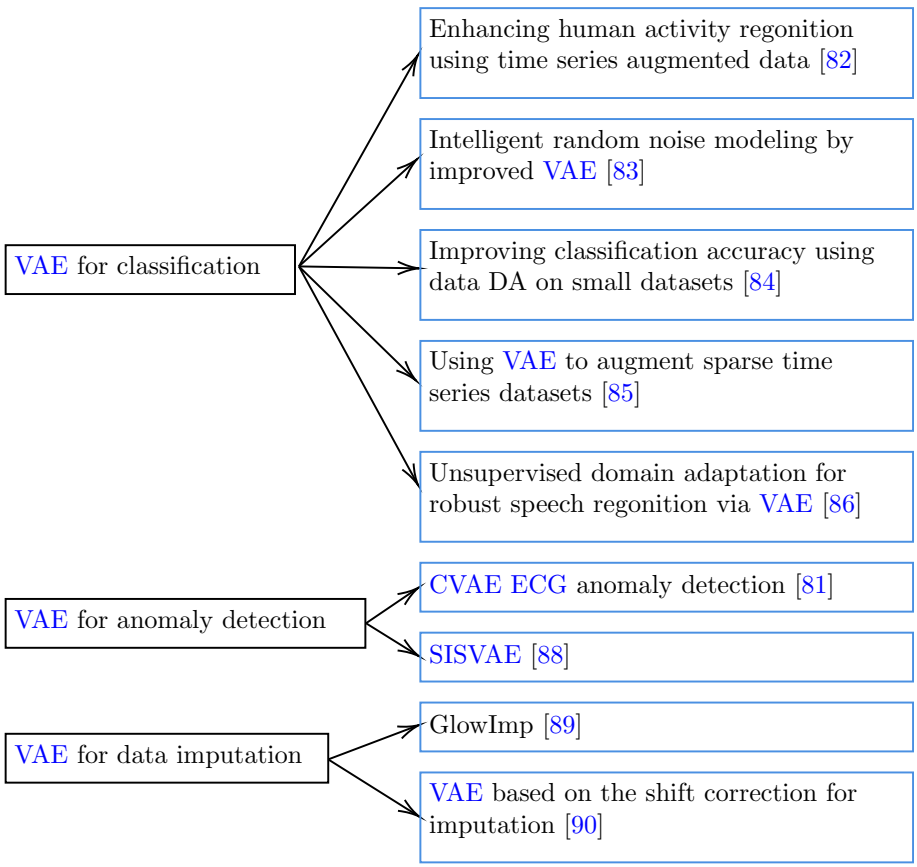


Fig. 5 Taxonomy of the presented variational autoencoder (VAE) architectures

usually used to fill gaps in temporal spaces where there are no available data. In this sense, VAE generates synthetic information on demand to fill these gaps, generating new information following the distribution of the original data.

The GlowImp architecture [89] was proposed as a combination of the Wasserstein GAN (WGAN) [93] architecture together with a VAE to impute missing data. The architecture is composed of the so-called Glow VAE, which incorporates a function that takes the latent distribution of the traditional VAE encoder and interpolates the missing values via the Glow Model. The other main part of the architecture is the GAN model where the generator corresponds with the decoder of the VAE and the discriminator forces the system to produce realistic samples. The results of the model are tested using two different datasets, the KDD Cup Challenge 2018 dataset containing air quality weather data and the PhysioNet Challenge 2012 which is a collection of multivariate clinical time series data. The architecture of the GlowImp can be seen in Fig. 6.

The work of Li et al. [90] presents a VAE architecture to impute temporal values using meteorological datasets. In order to fill in the missing values of the data samples, shift correction is used. This correction tries to counteract the deviation caused by the missing values. This correction is used in the Gaussian latent distribution, where a shift hyperparameter λ is applied which is manually set to centre the latent distribution, thus correcting the possible bias produced by missing values. The VAE architecture used in this work to impute the missing values is β- VAE [94].

6.3 Data Augmentation through GAN

GANs are one of the most popular generative models of the last decade, since its introduction in 2014 by Ian Goodfellow [3] this generative architecture has positioned itself as one of the main algorithms for DA. The main strength of the GAN architecture is that it learns the distribution of the data by extracting the main features of the samples, without copying the distribution directly. This is known as data generation and its main strength with respect to data augmentation is that it synthesises completely new samples, in contrast with other techniques where the original samples where transformed to generate new instances. This fosters the generalisation and creativity of the synthetic data generated by the model. It is also an important factor that the training of the networks is unsupervised, not necessarily to have labelled data to learn the distribution.

6.3.1 Taxonomy for the GAN algorithms reviewed

Fig. 7 shows a scheme to group the different research reviews in Sect. 4.3, this way all the GAN algorithms for DA can be viewed schematically.

6.3.2 Long Short-Term Memory (LSTM) based GAN

One of the approaches to adapt the GAN architecture to time series is to use recurrent networks as the base of artificial neural network (ANN). These GANs substitute the regular fully connected or convolutional layers with recurrent layers, able to have memory that links temporal features of the data. The main strength of this set of architectures is that they are able to process this temporal information that the input data have, similar to the spatial information processing of a convolutional neural network.

Continuous recurrent GAN (C-RNN-GAN) [58] is one of the first GAN architectures proposed specifically for time series data. In particular, using them to learn and synthesise music tracks was proposed in this work. This GAN uses LSTM blocks [92] as its main learning structure. The learning algorithm is the same as standard training GAN, where the network generator concatenates each input with the output of the previous cells and the discriminator is made up of a bidirectional recurrent network [35]. The internal composition of the discriminator is based on the work of Horchreiter [100] and Bengio et al. [101] that avoids the gradient vanishing and strengths the temporal dependencies. The results of the work are discussed using a dataset of 3697 musical midi files from 160 different composers of classical music with a tick resolution of 384 per quarter note.

The work presented by Haradal et al. [95] also proposes a GAN architecture based on the implementation of LSTM

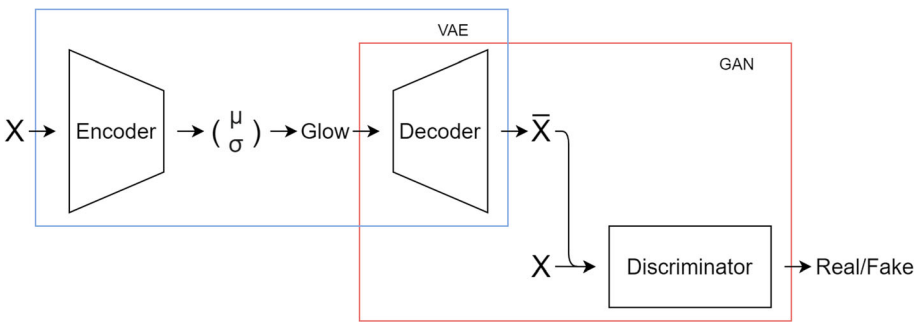


Fig. 6 GlowImp architecture. Based on the figure of [89]

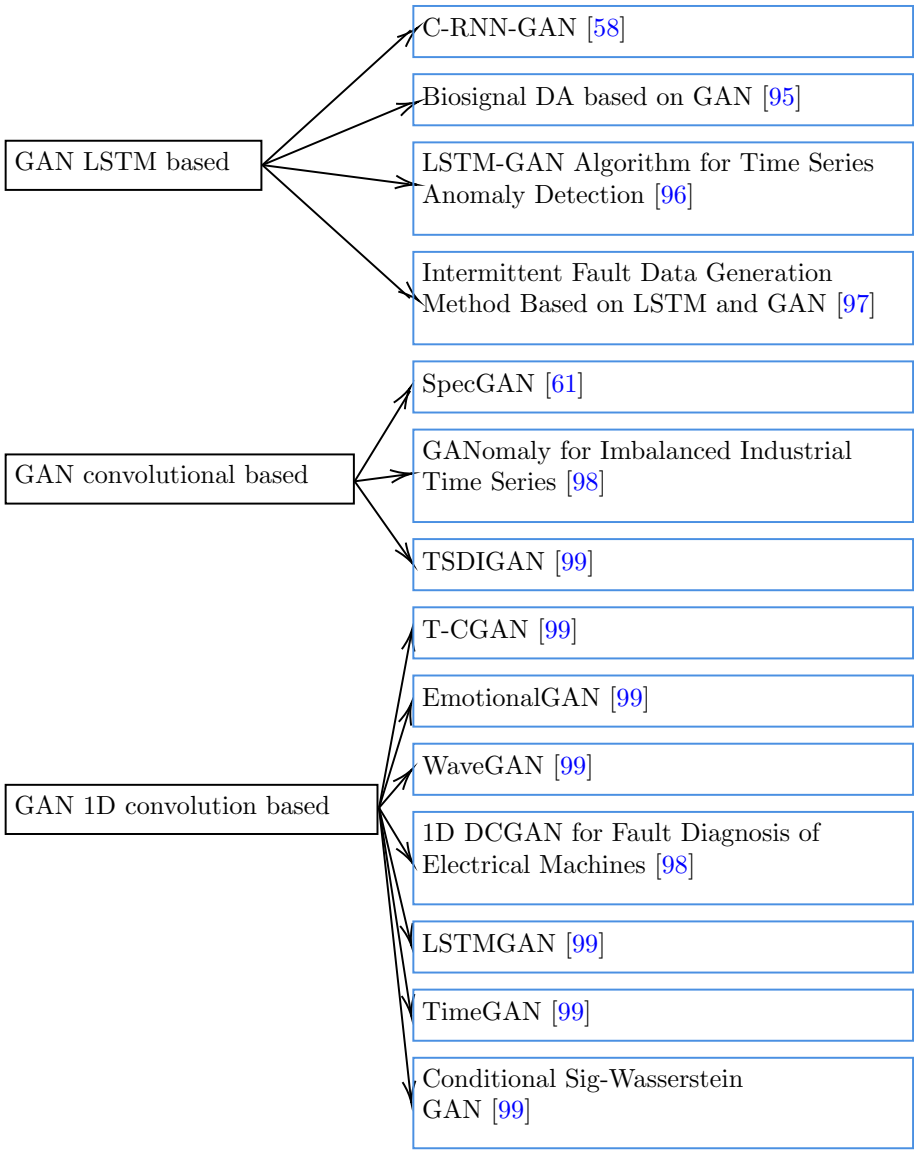


Fig. 7 Taxonomy of the presented GAN architectures

cells in both the generator and discriminator networks to adapt to time series data. The discriminator output is generated by applying an average pooling to the outputs generated by each layer, averaging the whole data sample into a unique scalar output which corresponds to the probability of the sample being generated by the generator network. This architecture was used to generate electrocardiogram (ECG) [102] and electroencephalogram (EEG) [103] data to improve the classification accuracy of an artificial neural network (ANN) classifier.

The LSTM and GAN combination has also been used for anomaly detection in the work of Zhu et al. [96] where the LSTM layers are used in the discriminator to extract temporal information from the data, while the GAN architecture provides the system with the ability to extract the most important features of the data. Training for detecting anomalies in the data has two phases. The first phase, known as the training phase, is a standard GAN training in which the discriminator learns how to distinguish between real and synthetic data. In the second phase, the so-called testing phase, the training consists of a feature extraction that generates and embedding of the dataset samples, these features are then reconstructed by the generator and compared with the original data, the task of the discriminator is to distinguish the real and the reconstructed data, which is anomalous. This research tested its results using two different datasets, electrocardiogram (ECG) data with a window of 96 data values that is training to detect anomalous cases and a dataset with the statistics of taxi traffic in New York City, with 48 points each data sample.

The work presented by Shi et al. [97] uses the GAN architecture to generate sequences of faulty data from two different types. Different models are trained for each type. The generator and discriminator of each GAN are made of a many-to-many LSTM model that processes the voltage signal data and the sampling length of each step of the sequence. In this way, the generator output is composed of two vectors, one for the voltage and the other for the length, while the discriminator processes these data and its output is generated by averaging the classification of each step and generating a unique binary output.

6.3.3 Convolutional GAN applied to the time series domain

In order to apply GAN to replicate time series, one of the most popular techniques used is to treat the time data as an image. Different approximations have been used in this field, where the focus is on how to transform the data into an image format, rather than adapting the GAN architecture to process time series information. One of the main advantages of this technique is that it does not have to deal with the design of GAN, which is a complex process due to the particularities of the architecture. The adaptation of the original data to an image is different in each case. Different works published during the last years will be reviewed in order to study different approximations to this transformation.

An example of this use is the one proposed with SpecGAN [61] which tries to operate with sound spectrograms that represent audio samples. This approach uses deep convolutional GAN (DCGAN) [44] as the main algorithm for DA, but prior to that, it processes the audio signal to generate images for each audio track. The process of transforming audio into image "can be approximately inverted" in the author's own words. First, the Fourier transformation is applied to each audio to generate a matrix of the frequencies of the data. Then, the scale of the data is adapted logarithmically and normalised to a normal distribution for a better understanding. Finally, the images are clipped to 3 standard deviations and rescaled within the $[-1, 1]$ range. In particular, the 16384 points of each sample are converted into a 128x128 pixel image. As mentioned above, this process is reversible, so once the new data are generated using deep convolutional GAN (DCGAN) they can be transformed to audio data using the reverse process. One advantage of using this process is that it opens up the possibility of comparing different audio generation algorithms treating the results as images; in the original paper, the results of the SpecGAN are compared with the WaveGAN, which is proposed in the same article.

The work presented by Jiang et al. [98] uses the GANomaly architecture [104] to process different time series data. The GANomaly is used for anomaly detection in industrial tasks; it introduces a feature extraction into the network, which pre-processes the input data of both the generator and the generator. The generator is composed of an encoder–decoder–encoder network, which makes it possible to learn the latent representations generated by the feature extraction part. Regarding the data used for training, rolling bearing data was used to detect anomalies, collected at 12–48 kHz in two different datasets. The collected data is converted from the time series domain into images by generating a spectrogram, thus converting the time series data into the image domain. In particular, they used Bearing Data from Case Western Reserve University.[1]

The Traffic Sensor Data Imputation GAN (TSDIGAN) [99] is an architecture proposed for missing data reconstruction. In particular, traffic data is used consisting on 104,544 traffic records. In this work, GAN is in charge of generating synthetic data that fill in the missing data gaps with realistic information. The approach used in the paper to treat time series traffic data is to transform them into an image format using the proposed method called Gramian Angular Summation Field (GASF). The Gramian Angular Summation Field (GASF) algorithm is focused on maintaining the time dependency of the traffic data; this algorithm is capable of transforming the data into a matrix by representing each time data point to a polar coordinate system within the range $[-1, 1]$. Then, each point is encoded by its angular cosine and radius. This generates a matrix with the temporal correlation between each point, which is then fed to the networks. Finally, the data are processed using a convolutional-based GAN that uses its generator to generate new data and reconstruct the missing values.

6.3.4 1D convolutional GAN

Temporal cnn are cnn [105] where the convolutional operation is calculated in 1D instead of traditional 2D convolution. These networks adapt the geometric information captured by the 2D cnn to a temporal domain, lowering the dimensions of the learnt filters to 1D. These networks have been used in works such as [15] to classify data from temporal series.

In the recent years, different GAN architectures have been proposed that use these 1D convolutional layers as a base, replacing the traditional 2D convolutions of GAN applied to computer vision tasks. In this approximation, it is very straightforward to adapt traditional GAN architectures to the time series domain, making it very plausible for use in time series-related tasks.

[1] https://engineering.case.edu/bearingdatacenter.

The temporal-conditional GAN (T-CGAN) [106] is a GAN architecture based on the idea of transforming the Conditional GAN (CGAN) [107] architecture to time series domain by replacing the 2D convolutional layers with the 1D convolutional layers. The performance of the model was validated using one synthetic and three real-world datasets. The synthetic data was constructed using sine waves and sawtooth waves. The other datasets consisted of: astronomical light curves of 1024 points per sample, a power demand dataset of samples of 24 points, and an electrocardiogram (ECG) dataset with 96 points for each sample.

Emotional GAN [108] also applies these 1D convolutional layers to create a GAN architecture to augment an electrocardiogram (ECG) dataset improving the classification of support vector machine (SVM) and random forest models when classifying the emotions of each subject. This work used different datasets varying their frequency rate between 256 and 2048 Hz.

The work published by Donahue et al. [61] presents the WaveGAN architecture, which is based on the application of 1D convolutional layers to sound data. This GAN uses the deep convolutional GAN (DCGAN) architecture, but changes the convolutions to 1D. As suggested, these 1D convolutions should have a wider receptive field respecting the 2D convolutions of image processing; this is based on the particularities of the audio data, in which each cycle of a musical note sampled at 16 kHz may take 36 samples to complete. Therefore, it is necessary to use wider filters to capture the distanced temporal dependencies of the data. This feature of the sound data was previously taken into account with solutions such as the dilated convolutions proposed in WaveNet [60]. This enlargement of the receptive field is compensated for by reducing one dimension, changing from 5x5 convolutions to 25 1-dimensional convolutions and maintaining the number of parameters of the network. The rest of the architecture maintains the standard GAN architecture, allowing the synthesis of audio tracks with unsupervised training GAN.

This approximation has also been followed by Sabir et al. [109] for augmenting DC current signals, using samples of current signal with a frequency of 100 Hz during 16 s. The proposed work used the deep convolutional GAN (DCGAN) architecture as a base and changes the original convolutions to 1D convolutions. In particular, this work has two different GANs, one that generates healthy signals and the other is in charge of generating faulty data.

There are also hybrid implementations that combine 1D convolutions with other techniques, such as in [96] where LSTM-GAN is proposed. This architecture combines the LSTM cell in the discriminator network with the 1D convolutional layers used in the generator network.

6.3.5 Time series Generative Adversarial Networks (TimeGAN)

The TimeGAN architecture [55] tries to implement a GAN model to perform DA on time series data, but differentiates itself from other previous alternatives by adding a new loss function that tries to capture the stepwise dependencies of the data. Previous implementations of GAN in data sequences were based on the use of recurrent networks for the generator and discriminator networks of GAN [56, 58], but this approximation may not be sufficient to accurately replicate the temporal transitions of the original data.

This work divides the data features into two different classes: static features and temporal features. Static features $\mathbf{S}$ do not vary over time, e.g., gender, while temporal features $\mathbf{X}$ change. In other words, the static features are characteristics of the data that are not directly related to the time series sample but capture important properties of it.

The TimeGAN is tested in four different datasets. First, a synthetic dataset is generated using sinusoidal sequences with an average length of 24 points. A stock prices time series dataset from 2004 to 2019 is used with an average length of 24 days. Third, a dataset of energy prediction with samples of 24 h of length on average is used. Finally, a medical lung cancer events dataset is used with an average of 58 events per sample.

The proposed architecture adds, in addition to the generator and discriminator networks, two new networks: the encoder and recovery networks. These networks are responsible for embedding the input data in the latent space, as an autoencoder [110] would traditionally do. This system learns the so-called embedding and recovery functions to take the static and temporal features into two separate latent codes $\mathbf{h}_s$ and $\mathbf{h}_t$ and recover the input information $\mathbf{S}$ and $\mathbf{X}$.

The generator and discriminator parts of the network do the same work as they would do in a traditional GAN, using the discriminator to differentiate between real and synthetic samples. But in this case, the generator generates the data for the embedding space, while the discriminator also takes this embedding as input for its classification.

The main innovation of TimeGAN is implemented in the generator, which, in addition to the normal generation of synthetic samples, is also forced to learn the stepwise dependencies of the data. To do so, the generator receives as input the synthetic embedding $\mathbf{h}_s, \mathbf{h}_{t-1}$ and computes the next vector $\mathbf{h}_s, \mathbf{h}_t$. This new function is learned by a new supervised loss function that compares the generator forecast with the real data.

Therefore, the training objectives of the presented architecture can be divided into 3 different loss functions.

- *Reconstruction loss* ($\mathcal{L}_R$) This loss is used in the reversible mapping part of the network, composed of the encoder and recovery networks. The length T of each sequence is also a random variable, the distribution of which, for notational convenience, it will be absorbed into distribution p. It is given by:

$$\mathcal{L}_R = \mathbb{E}_{\mathbf{s},\mathbf{x}_{1:T}\sim p}\left[\|\mathbf{s}-\tilde{\mathbf{s}}\|_2 + \sum_t \|\mathbf{x}_t - \tilde{\mathbf{x}}_t\|_2\right], \tag{13}$$

  where the tilde denotes the reconstructed samples and $||\cdot||$ stands for the standard (Euclidean) norm.
- *Unsupervised loss* ($\mathcal{L}_U$) In TimeGAN, the generator has two different types of input during training. First, the generator receives synthetic embeddings $\hat{h}_S, \hat{h}_{1:t-1}$, which are autoregressive, to generate the next synthetic vector $\hat{h}_t$. In this process, the gradients are computed under unsupervised loss. This is as expected, that is, to allow maximising D or minimising G the probability of providing the correct classifications $\hat{y}_S$, $\hat{y}_{1:T}$ for both training data $\mathbf{h}_s, \mathbf{h}_{1:T}$ and synthetic output $\hat{h}_S, \hat{h}_{1:T}$ from the generator. The unsupervised loss function is the equivalent loss function of a normal GAN that attempts to distinguish real and fake samples. It is given by:

$$\begin{aligned}\mathcal{L}_U = {} & \mathbb{E}_{\mathbf{s},\mathbf{x}_{1:T}\sim p}[\log y_S + \sum_t \log y_t] \\ & + \mathbb{E}_{\mathbf{s},\mathbf{x}_{1:T}\sim \hat{p}}[\log(1-\hat{y}_S) + \sum_t \log(1-\hat{y}_t)]\end{aligned} \tag{14}$$

  where y_s and y_t are the classification of the discriminator for static and temporal features and the accent denotes synthetic samples.
- *Supervised loss* ($\mathcal{L}_S$) To encourage the generator to learn the conditional transitions of the data, this function is designed that measures the similarity between the real and the synthetic samples created by the generator when applying the forecasting. The loss function is denoted as follows:

$$\mathcal{L}_S = \mathbb{E}_{\mathbf{s},\mathbf{x}_{1:T}\sim p}\left[\sum_t \|\mathbf{h}_t - g_{\mathcal{X}}(\mathbf{h}_S, \mathbf{h}_{t-1}, \mathbf{z}_t)\|_2\right] \tag{15}$$

  where $g_{\mathcal{X}}$ denotes the sample synthesised by the generator, taking as input the embedded anterior sample $\mathbf{h}_S, \mathbf{h}_{t-1}, \mathbf{z}_t$. An overview of the learning scheme of TimeGAN can be seen in Fig. 8.

6.3.6 Conditional Sig-Wasserstein GAN

The Conditional Signature Wasserstein GAN [111] was proposed as a method of maintaining long temporal dependencies in time series data. Regarding the previous models, this architecture is able to outperform previous models such as TimeGAN providing better synthetic data for DA.

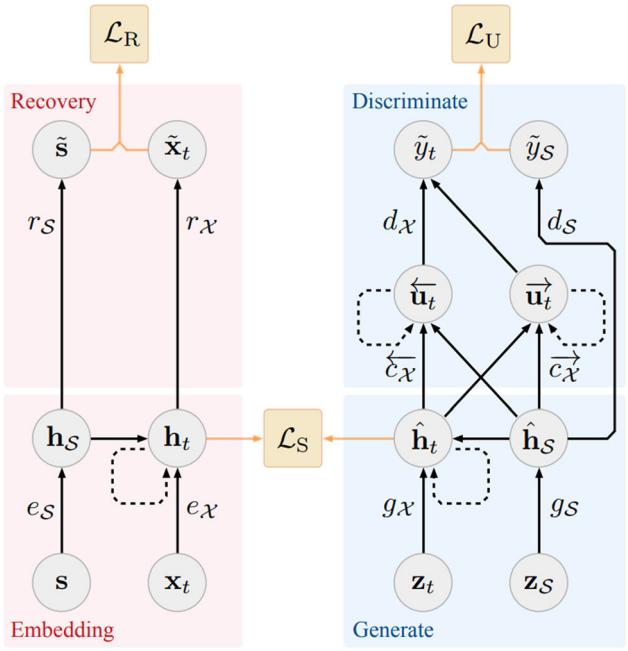


Fig. 8 TimeGAN architecture. Extracted from [55]

In this paper, it is proposed a new metric for evaluating the properties of a data stream, providing a description of the sample. This metric is used in the discriminator (D) network to differentiate real and synthetic samples. The Sig-Wasserstein metric measures the path space of the data using the Wasserstein distance. In this case, the main strength of this method is that it simplifies the training by replacing a neural network discriminator (D) for a linear regression using the Sig-Wasserstein distance. This process eliminates the cost of approximating a discriminator (D).

The results of the model are tested using different stock market datasets to predict the close prices and the volatility of different actives.

As a Generator (G) network, it is used an AR-FNN generator, which is able of capturing temporal dependencies of time series data.

Figure 9 shows a scheme of the training process of the Conditional Sig-Wasserstein GAN. As it can be seen, the real and fake samples can be distinguished by using the Sig-Wasserstein metric.

6.4 Data Augmentation (DA) based on Dynamic Time Warping (DTW)

6.4.1 Dynamic Time Warping (DTW) Barycenter Averaging

Dynamic time warping (DTW) [112] is a classical algorithm that measures the similarity between two data

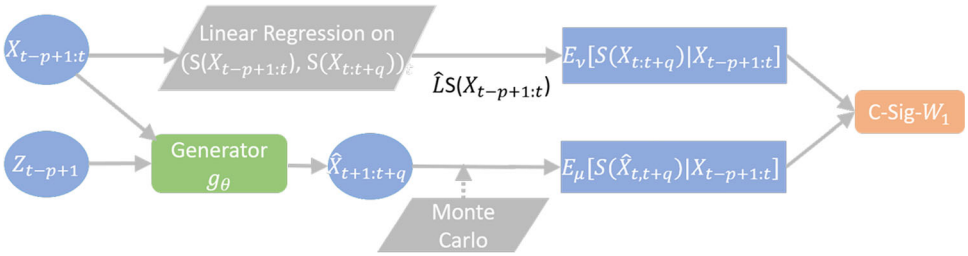


Fig. 9 Conditional Sig-Wasserstein GAN training. Extracted from [111]

sequences. This method was used as a base in [113], where the effectiveness of the new generated synthetic time series data was evaluated using augmented training sets for time series classification through the 1-NN classifier in conjunction with dynamic time warping (DTW). In particular, 85 datasets of the UCR archive [62] were used in the experiments. The idea is to manipulate the distribution manifold to generate infinite new samples of data. They achieve this by changing the weights of a set of time series, such as the set $D = \{(T_1, w_1), \ldots, (T_N, w_n)\}$ is embedded in a space E and the average of dynamic time warping (DTW) is denoted as follows:

$$\arg\min \bar{T} \in E \sum_{i=1}^{N} w_i \cdot \mathrm{DTW}^2(\bar{T}, T_i) \tag{16}$$

where w is the weight of each sample.

To calculate $\bar{T}$, they use the expectation–maximisation algorithm and to decide the weight values, three different methods are proposed:

- *Average all* This method generates the weight vector values using a flat Dirichlet distribution. The main problem with this method is that it tends to fill in data spaces where it should not.
- *Average selected* This method focusses on selecting a subset of close samples. Thus, it prevents empty spaces from being filled with information because the subsets of samples are close together in the manifold.
- *Average selected with distance* The difference between this method and the previous one is that this method calculates the relative distance between the near samples of data.

6.4.2 Suboptimal element alignment averaging

SuboPtimAl Warped time series geNEratoR (SPAWNER) [114] is a DA method based on the dynamic time warping (DTW) algorithm [112]. The dynamic time warping (DTW) algorithm is used in this DA method to align different multidimensional signals X_1, X_2, giving the so-called *warping path* W which is a sequence of points that minimises the distance between these input signals. The results of the model were tested using two different electrocardiogram (ECG) datasets.

SPAWNER algorithm takes the warping path calculated with the dynamic time warping (DTW) algorithm and introduces a new random element to the sequence, known as w_p. This new point is generated using a uniformly distributed random number within the range (0, 1). Then, the new optimal path is forced to contain the new generated element, obtaining the new warping paths W_1^*, W_2^*. Both sequences are aligned using a parameter called ξ, which reduces the flexibility of the path. Finally, both warp paths are concatenated, generating the path $W_{1,2}^*$ from which the new time series signals x_1^*, x_2^* are obtained.

It is observed that for some multivariate signals, this variation of DA is not enough. Therefore, a random variance is also applied to each point of the signal using a normal distribution such as $x^* \sim N(\mu, \sigma^2), \mu = 0.5(x_1^* + x_2^*), \sigma = 0.05|x_1^* - x_2^*|$.

The use of different alignment methods for text or image data is also proposed, instead of using dynamic time warping (DTW) which is proposed when signals are used. Therefore, the overall algorithm can be easily translated to other domains, with the need for an alignment method between two samples.

7 Discussion

Data augmentation algorithms in the time series domain are really important for improving the available datasets, whose creation is not always easy. In general terms, all the methods presented in this work are algorithms specifically designed for data augmentation (DA) in time series but, in other cases, they are usually adaptations of architectures that were originally designed for other domains, such as image processing. However, the GAN-based algorithms themselves have their beginnings in the field of imaging and have gradually been integrated into other areas.

Regarding the length of the data that each algorithm can process, it should be noted the variety of sizes and dataset types of the different reviewed researches. One of the main strengths that artificial intelligence algorithms have is that, due to the fact that their learning is based on the particular

data of each application, they can operate with almost any source of data. This leads to a great variation in the size of the time series windows that most of the algorithms use. In some cases, the data are even processed to treat it as an image, which is symptomatic of the flexibility of DL algorithms. In the case of traditional algorithms, they can also process any length of data if it is properly adapted.

In this section, it will be analysed the main advantages and disadvantages of each type of algorithm.

7.1 Advantages

Traditional algorithms are widely developed and studied so that their results can be fairly compared. In DA, they allow you to work by modifying the examples already present, which allows you to control variations. In addition, the simplicity of the algorithms themselves by greatly reducing the number of hyperparameters to be configured results in less time to set them up and the need for less data to train them.

Second, the VAE generative algorithms allow one to control to a greater extent the variability of the generated data by directly influencing the standard deviation of the latent distribution of the original dataset. This feature allows, among all algorithms, the greatest control of the variability of the generated data. VAE is commonly used for anomaly detection cases due to its better performance.

Finally, the most current generative models are a breakthrough in the area due to their great results. GANs, like VAEs, allow synthetic data to be generated and, at the cost of losing some control over data generation, they are algorithms capable of much better generalisation. All this is due to the training scheme itself, which allows GANs models to learn the distribution that follows the original dataset and, through it, generate synthetic data according to the distribution of the dataset.

Furthermore, since GANs are relatively recent algorithms, they benefit from greater attention from the scientific community, which means that there is more recent research focused on improving their results than other algorithms.

7.2 Disadvantages

In terms of limitations, the use of traditional algorithms is quite limited because they are based on making modifications to elements of the real dataset. Therefore, they can often produce invalid examples. In general, they are limited to generate examples of lower quality and never to generating new elements. Normally, the reviewed algorithms require a pre-processing phase to normalise the input data, which can lead to more complex algorithms involving previous steps. Otherwise, the learning of the artificial neural network (ANN) would be inefficient [115, 116]

Although VAEs are algorithms capable of generating synthetic data, as opposed to traditional algorithms that only modify the original data, new neural network (NN) models such as GANs have mitigated their use in the field because by nature they are capable of generating fewer data than the most current generative networks. Despite this, because they can very precisely control the variability of the generated data, there are fields of application that still continue to use them.

Regarding GAN, it can be said that despite their great results, there are certain difficulties that slow down their progress. GANs are by far the most complex models currently available and, due to the particularities of the way they are trained, they are extremely difficult to train and obtain results.

GAN are one of the most difficult models to train. The main problems that these networks suffer are mode collapse [117, 118], instability [119], convergence evaluation [120] and evaluation metrics.

Due to the GAN instability problems [119, 121, 122], most of the time a great portion of the samples synthesised by the Generator (G) network lack of quality in certain aspects as the emergence of image artefacts [123]. In addition, the lack of convergence evaluation of the networks makes it hard to detect when the generated data is of high quality. Therefore, one of the main problems of GAN is that their results are not fully reliable.

7.3 Open issues and challenges

Some authors [124] tend to differentiate between DA and data generation due to the great advances made in neural network (NN) models. Traditional algorithms are always framed in the area of DA since the data they produce are always based on existing data; as an open problem, they generate less varied data but more control over what is generated. Furthermore, data generation algorithms produce new data so aggressively that much of the generated data is not possible, degenerating the quality of the augmented dataset [125].

Unlike the limitation of the scarcity of data augmented with traditional models, AEs and VAEs are born to cover the deficiency of the generated data. In [126], they demonstrate the capability of generative neural network (NN) models to add more diversity to the dataset. In addition, traditional algorithms tend not to be flexible in taking a trained model and applying it to another problem, forcing a rethink of the algorithm. Neural networks, in this aspect, tend to be more flexible, and able to use the same trained model in different problems. In [106], T-CGAN (Sect. 6.3.4) where different datasets are exposed with the

same architecture, or in [96], LSTM-GAN that uses as inputs datasets as disparate as one made of electrocardiograms and another comprising taxi statistics.

However, although generative models offer great advantages, GANs have significant additional problems, especially in training. Typical problems such as modal collapse, Nash equilibria, gradient vanishing or instability are suffered in every training of these models, making their optimisation a very complex process [127, 128].

In general, all generative models share the same open problem that often complicates their validation process. As shown in Sect. 5, despite the existence of some evaluation metrics, there is no consensus in the community on which should be used. For example, in [55] authors use empirical evaluation for data generation, but for visualisation they use PCA and a discriminative and a predictive model to see how they have improved after adding the synthetic images. In [61], authors propose the Inception Score, a measure of Nearest Neighbour and empirical measurement by humans, and in [99], traditional measures of deep learning (MAE, RMSE and MRE) to compare the generation of future values are used. If the focus is also put on GAN models, it must be taken into account that, to this problem, there is no method for these architectures to define what the stop condition is in a training.

8 Conclusion

Due to the significant evolution that DA has undergone in recent years, more and more fields are emerging in which to apply and improve the results. This article is focused on giving a comprehensive overview of the main algorithms used for data augmentation (DA) in the field of time series. The review is organised in a taxonomy, consisting of basic and advanced approaches, where it is summarised representative methods of each algorithm (traditional, VAEs and GANs) comparing them empirically, disaggregate by application areas and highlight advantages/disadvantages for future research.

Acknowledgements This work was supported by the Comunidad de Madrid under Convenio Plurianual with the Universidad Politécnica de Madrid in the actuation line of Programa de Excelencia para el Profesorado Universitario. The third-named author has been partially supported by the Madrid Government (Comunidad de Madrid—Spain) under the Multiannual Agreement with the Universidad Complutense de Madrid in the line Research Incentive for Young PhDs, in the context of the V PRICIT (Regional Programme of Research and Technological Innovation) through the project PR27/21-029 and by the Ministerio de Ciencia e Innovación Project PID2021-124440NB-I00 (Spain).

Funding Open Access funding provided thanks to the CRUE-CSIC agreement with Springer Nature.

Data availability No datasets were generated or analysed during the current study, and all figures are taken from the papers listed in the references or are generated by us.

Declarations

Conflict of interest All authors declare that there is no conflict of interest in this paper.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Duong H-T, Nguyen-Thi T-A (2021) A review: preprocessing techniques and data augmentation for sentiment analysis. Comput Soc Netw 8(1):1–16
2. Felix EA, Lee SP (2019) Systematic literature review of preprocessing techniques for imbalanced data. IET Softw 13(6):479–496
3. Goodfellow IJ, Pouget-Abadie J, Mirza M, Xu B, Warde-Farley D, Ozair S, Courville A, Bengio Y (2014) Generative adversarial networks
4. Lecun Y (1987) PhD Thesis: Modeles connexionnistes de L'apprentissage (connectionist Learning Models). Universite P. et M. Curie (Paris 6)
5. Kingma DP, Welling M (2014) Auto-encoding variational bayes
6. Navidan H, Moshiri PF, Nabati M, Shahbazian R, Ghorashi SA, Shah-Mansouri V, Windridge D (2021) Generative adversarial networks (gans) in networking: a comprehensive survey & evaluation. Comput Netw 194:108149
7. Rigaki M, Garcia S (2018) Bringing a gan to a knife-fight: adapting malware communication to avoid detection. In: 2018 IEEE security and privacy workshops (SPW), pp 70–75. IEEE
8. Mozo A, González-Prieto Á, Pastor A, Gómez-Canaval S, Talavera E (2022) Synthetic flow-based cryptomining attack generation through generative adversarial networks. Sci Rep 12(1):1–27
9. Wang Q, Farahat A, Gupta C, Zheng S (2021) Deep time series models for scarce data. Neurocomputing 456:504–518
10. Cao L, Horn S, von Ehrenheim V, Anselmo Stahl R, Landgren H (2022) Simulation-informed revenue extrapolation with confidence estimate for scaleup companies using scarce time-series data. In: Proceedings of the 31st ACM international conference on information & knowledge management, pp. 2954–2963
11. Bhowmik AK, Cabral P (2015) Spatially shifting temporal points: estimating pooled within-time series variograms for scarce hydrological data. Hydrol Earth Syst Sci Discuss 12(2):2243–2265
12. Chlap P, Min H, Vandenberg N, Dowling J, Holloway L, Haworth A (2021) A review of medical image data

augmentation techniques for deep learning applications. J Med Imaging Radiat Oncol 65(5):545–563
13. Naveed H (2021) Survey: image mixing and deleting for data augmentation. arXiv preprint arXiv:2106.07085
14. Feng SY, Gangal V, Wei J, Chandar S, Vosoughi S, Mitamura T, Hovy E (2021) A survey of data augmentation approaches for nlp. arXiv preprint arXiv:2105.03075
15. Iwana BK, Uchida S (2021) An empirical survey of data augmentation for time series classification with neural networks. PLoS ONE 16(7):0254841
16. Brophy E, Wang Z, She Q, Ward T (2021) Generative adversarial networks in time series: a survey and taxonomy. arXiv preprint arXiv:2107.11098
17. Patil AG, Surve A, Gupta AK, Sharma A, Anmulwar S (2016) Survey of synthetic traffic generators. In: 2016 international conference on inventive computation technologies (ICICT), vol. 1, pp. 1–3. IEEE
18. Abufadda M, Mansour K (2021) A survey of synthetic data generation for machine learning. In: 2021 22nd international arab conference on information technology (ACIT), pp. 1–7. IEEE
19. Gao N, Xue H, Shao W, Zhao S, Qin KK, Prabowo A, Rahaman MS, Salim FD (2022) Generative adversarial networks for spatio-temporal data: a survey. ACM Trans Intell Syst Technol (TIST) 13(2):1–25
20. Raghunathan TE (2021) Synthetic data. Annu Rev Stat Appl 8:129–140
21. Wen Q, Sun L, Yang F, Song X, Gao J, Wang X, Xu H (2020) Time series data augmentation for deep learning: A survey. arXiv preprint arXiv:2002.12478
22. Lim B, Zohren S (2021) Time-series forecasting with deep learning: a survey. Phil Trans R Soc A 379(2194):20200209
23. Krizhevsky A, Hinton G, et al (2009) Learning multiple layers of features from tiny images
24. Simonyan K, Zisserman A (2014) Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556
25. He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 770–778
26. Szegedy C, Liu W, Jia Y, Sermanet P, Reed S, Anguelov D, Erhan D, Vanhoucke V, Rabinovich A (2015) Going deeper with convolutions. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 1–9
27. Huang G, Liu Z, Van Der Maaten L, Weinberger KQ (2017) Densely connected convolutional networks. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 4700–4708
28. Mikołajczyk A, Grochowski M (2018) Data augmentation for improving deep learning in image classification problem. In: 2018 international interdisciplinary PhD workshop (IIPhDW), pp. 117–122. IEEE
29. Lashgari E, Liang D, Maoz U (2020) Data augmentation for deep-learning-based electroencephalography. J Neurosci Methods 346:108885
30. Ashfahani A, Pratama M, Lughofer E, Ong Y-S (2020) Devdan: deep evolving denoising autoencoder. Neurocomputing 390:297–314
31. Kullback S, Leibler RA (1951) On information and sufficiency. Ann Math Stat 22(1):79–86
32. Bandara K, Hewamalage H, Liu Y-H, Kang Y, Bergmeir C (2021) Improving the accuracy of global forecasting models using time series data augmentation. Pattern Recogn 120:108148
33. Wang Z, Yan W, Oates T (2017) Time series classification from scratch with deep neural networks: a strong baseline. In: 2017 international joint conference on neural networks (IJCNN), pp. 1578–1585. IEEE
34. Reimers N, Gurevych I (2017) Optimal hyperparameters for deep lstm-networks for sequence labeling tasks. arXiv preprint arXiv:1707.06799
35. Schuster M, Paliwal KK (1997) Bidirectional recurrent neural networks. IEEE Trans Signal Process 45(11):2673–2681
36. Karim F, Majumdar S, Darabi H, Chen S (2017) Lstm fully convolutional networks for time series classification. IEEE Access 6:1662–1669
37. Isola P, Zhu J-Y, Zhou T, Efros AA (2017) Image-to-image translation with conditional adversarial networks. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 1125–1134
38. Yi Z, Zhang H, Tan P, Gong M (2017) Dualgan: Unsupervised dual learning for image-to-image translation. In: Proceedings of the IEEE international conference on computer vision, pp. 2849–2857
39. Zhu J-Y, Park T, Isola P, Efros AA (2017) Unpaired image-to-image translation using cycle-consistent adversarial networks. In: Proceedings of the IEEE international conference on computer vision, pp. 2223–2232
40. Wang L, Sindagi V, Patel V (2018) High-quality facial photo-sketch synthesis using multi-adversarial networks. In: 2018 13th IEEE international conference on automatic face & gesture recognition (FG 2018), pp. 83–90. IEEE
41. Yang C-HH, Tsai Y-Y, Chen P-Y (2021) Voice2series: Reprogramming acoustic models for time series classification. In: International conference on machine learning, pp. 11808–11819. PMLR
42. Wang Z, Yan W, Oates T (2017) Time series classification from scratch with deep neural networks: a strong baseline. In: 2017 International joint conference on neural networks (IJCNN), pp. 1578–1585. IEEE
43. Borji A (2019) Pros and cons of gan evaluation measures. Comput Vis Image Underst 179:41–65
44. Radford A, Metz L, Chintala S (2015) Unsupervised representation learning with deep convolutional generative adversarial networks. arXiv preprint arXiv:1511.06434
45. Karras T, Aila T, Laine S, Lehtinen J (2017) Progressive growing of gans for improved quality, stability, and variation. arXiv preprint arXiv:1710.10196
46. Karras T, Laine S, Aila T (2019) A style-based generator architecture for generative adversarial networks. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp. 4401–4410
47. Karras T, Aittala M, Laine S, Härkönen E, Hellsten J, Lehtinen J, Aila T (2021) Alias-free generative adversarial networks. Adv Neural Inf Process Syst 34:852–863
48. Zhu M, Pan P, Chen W, Yang Y (2019) Dm-gan: dynamic memory generative adversarial networks for text-to-image synthesis. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp. 5802–5810
49. Gao L, Chen D, Zhao Z, Shao J, Shen HT (2021) Lightweight dynamic conditional gan with pyramid attention for text-to-image synthesis. Pattern Recognit 110:107384
50. Hartmann KG, Schirrmeister RT, Ball T (2018) Eeg-gan: Generative adversarial networks for electroencephalograhic (eeg) brain signals. arXiv preprint arXiv:1806.01875
51. Wang S, Rudolph C, Nepal S, Grobler M, Chen S (2020) Partgan: privacy-preserving time-series sharing. In: International conference on artificial neural networks, pp. 578–593. Springer
52. Salimans T, Goodfellow I, Zaremba W, Cheung V, Radford A, Chen X (2016) Improved techniques for training GANs
53. Gurumurthy S, Sarvadevabhatla RK, Radhakrishnan VB (2017) DeLiGAN : Generative Adversarial Networks for Diverse and

Limited Data. In: 2017 IEEE conference on computer vision and pattern recognition. pp. 4941–4949
54. Heusel M, Ramsauer H, Unterthiner T, Nessler B, Hochreiter S (2017) Gans trained by a two time-scale update rule converge to a local nash equilibrium. Advances in neural information processing systems. In: Neural information processing systems (NIPS). **30**
55. Yoon J, Jarrett D, Van der Schaar M (2019) Time-series generative adversarial networks. In: Advances in neural information processing systems **32**
56. Esteban C, Hyland SL, Rätsch G (2017) Real-valued (medical) time series generation with recurrent conditional gans. arXiv preprint arXiv:1706.02633
57. Naaz F, Herle A, Channegowda J, Raj A, Lakshminarayanan M (2021) A generative adversarial network-based synthetic data augmentation technique for battery condition evaluation. Int J Energy Res 45(13):19120–19135
58. Mogren O (2016) C-rnn-gan: Continuous recurrent neural networks with adversarial training. arXiv preprint arXiv:1611.09904
59. Sutskever I, Martens J, Hinton GE (2011) Generating text with recurrent neural networks. In: ICML
60. Van Den Oord A, Dieleman S, Zen H, Simonyan K, Vinyals O, Graves A, Kalchbrenner N, Senior AW, Kavukcuoglu K (2016) Wavenet: a generative model for raw audio. SSW 125:2
61. Donahue C, McAuley J, Puckette M (2018) Adversarial audio synthesis. arXiv preprint arXiv:1802.04208
62. Dau HA, Keogh E, Kamgar K, Yeh C-CM, Zhu Y, Gharghabi S, Ratanamahatana CA, Yanping Hu B, Begum N, Bagnall A, Mueen A, Batista G, Hexagon-ML (2018) The UCR time series classification archive. https://www.cs.ucr.edu/~eamonn/time_series_data_2018/
63. Fu B, Kirchbuchner F, Kuijper A (2020) Data augmentation for time series: traditional vs generative models on capacitive proximity time series. In: Proceedings of the 13th ACM international conference on pervasive technologies related to assistive environments, pp. 1–10
64. Debnath A, Waghmare G, Wadhwa H, Asthana S, Arora A (2021) Exploring generative data augmentation in multivariate time series forecasting: opportunities and challenges. Solar-Energy 137:52–560
65. Cazelles E, Robert A, Tobar F (2020) The wasserstein-fourier distance for stationary time series. IEEE Trans Signal Process 69:709–721
66. Cao P, Li X, Mao K, Lu F, Ning G, Fang L, Pan Q (2020) A novel data augmentation method to enhance deep neural networks for detection of atrial fibrillation. Biomed Signal Process Control 56:101675
67. Pan J, Tompkins WJ (1985) A real-time qrs detection algorithm. IEEE Trans Biomed Eng 3:230–236
68. Flores A, Tito-Chura H, Apaza-Alanoca H (2021) Data augmentation for short-term time series prediction with deep learning, pp. 492–506
69. Rashid KM, Louis J (2019) Window-warping: a time series data augmentation of imu data for construction equipment activity identification. In: ISARC. Proceedings of the international symposium on automation and robotics in construction, vol. 36, pp. 651–657. IAARC Publications
70. Um TT, Pfister FM, Pichler D, Endo S, Lang M, Hirche S, Fietzek U, Kulić D (2017) Data augmentation of wearable sensor data for parkinson's disease monitoring using convolutional neural networks. In: Proceedings of the 19th ACM international conference on multimodal interaction, pp. 216–220
71. Adachi S, Takemoto H, Kitamura T, Mokhtari P, Honda K (2007) Vocal tract length perturbation and its application to male-female vocal tract shape conversion. J Acoust Soc Am 121(6):3874–3885
72. Cui X, Goel V, Kingsbury B (2015) Data augmentation for deep neural network acoustic modeling. IEEE/ACM Trans Audio, Speech, Lang Process 23(9):1469–1477
73. Ko T, Peddinti V, Povey D, Khudanpur S (2015) Audio augmentation for speech recognition. In: 16th annual conference of the international speech communication association
74. Jaitly N, Hinton GE (2013) Vocal tract length perturbation (vtlp) improves speech recognition. In: Proceedings of ICML workshop on deep learning for audio, speech and language, vol. 117
75. Park DS, Chan W, Zhang Y, Chiu C-C, Zoph B, Cubuk ED, Le QV (2019) Specaugment: a simple data augmentation method for automatic speech recognition. arXiv preprint arXiv:1904.08779
76. Jeong CY, Shin HC, Kim M (2021) Sensor-data augmentation for human activity recognition with time-warping and data masking. Multimed Tools Appl 80(14):20991–21009
77. Le Guennec A, Malinowski S, Tavenard R (2016) Data augmentation for time series classification using convolutional neural networks. In: ECML/PKDD workshop on advanced analytics and learning on temporal data
78. Fawaz HI, Forestier G, Weber J, Idoumghar L, Muller P-A (2018) Data augmentation using synthetic data for time series classification with deep residual networks. arXiv preprint arXiv:1808.02455
79. Pan Q, Li X, Fang L (2020) Data augmentation for deep learning-based ecg analysis, 91–111
80. Lee H, Hwang S, Shin J (2019) Rethinking data augmentation: self-supervision and self-distillation. arxiv 2019. arXiv preprint arXiv:1910.05872
81. Al Nazi Z, Biswas A, Rayhan MA, Abir TA (2019) Classification of ecg signals by dot residual lstm network with data augmentation for anomaly detection. In: 2019 22nd international conference on computer and information technology (ICCIT), pp. 1–5. IEEE
82. Alawneh L, Alsarhan T, Al-Zinati M, Al-Ayyoub M, Jararweh Y, Lu H (2021) Enhancing human activity recognition using deep learning and time series augmented data. J Ambient Intell Humaniz Comput 12(12):10565–10580
83. Feng Q, Li Y, Wang H (2021) Intelligent random noise modeling by the improved variational autoencoding method and its application to data augmentation. Geophysics 86(1):19–31
84. Moreno-Barea FJ, Jerez JM, Franco L (2020) Improving classification accuracy using data augmentation on small data sets. Expert Syst Appl 161:113696
85. Goubeaud M, Joußen P, Gmyrek N, Ghorban F, Schelkes L, Kummert A (2021) Using variational autoencoder to augment sparse time series datasets. In: 2021 7th international conference on optimization and applications (ICOA), pp. 1–6. IEEE
86. Hsu W-N, Zhang Y, Glass J (2017) Unsupervised domain adaptation for robust speech recognition via variational autoencoder-based data augmentation. In: 2017 IEEE automatic speech recognition and understanding workshop (ASRU), pp. 16–23. IEEE
87. Demir S, Mincev K, Kok K, Paterakis NG (2021) Data augmentation for time series regression: applying transformations, autoencoders and adversarial networks to electricity price forecasting. Appl Energy 304:117695. https://doi.org/10.1016/j.apenergy.2021.117695
88. Li L, Yan J, Wang H, Jin Y (2020) Anomaly detection of time series with smoothness-inducing sequential variational auto-encoder. IEEE Trans Neural Netw Learn Syst 32(3):1177–1191
89. Liu C, Zhou H, Sun Z, Cui G (2021) Glowimp: combining glow and gan for multivariate time series imputation. In: International

conference on algorithms and architectures for parallel processing, pp. 50–64. Springer
90. Li J, Ren W, Han M (2021) Variational auto-encoders based on the shift correction for imputation of specific missing in multivariate time series. Measurement 186:110055
91. Sohn K, Lee H, Yan X (2015) Learning structured output representation using deep conditional generative models. Advances in neural information processing systems **28**
92. Hochreiter S, Schmidhuber J (1997) Long short-term memory. Neural Comput 9(8):1735–1780
93. Arjovsky M, Chintala S, Bottou L (2017) Wasserstein generative adversarial networks. In: International conference on machine learning, pp. 214–223. PMLR
94. Higgins I, Matthey L, Pal A, Burgess C, Glorot X, Botvinick M, Mohamed S, Lerchner A (2016) beta-vae: learning basic visual concepts with a constrained variational framework. In International conference on learning representations.
95. Haradal S, Hayashi H, Uchida S (2018) Biosignal data augmentation based on generative adversarial networks. In: 2018 40th annual international conference of the IEEE engineering in medicine and biology society (EMBC), pp. 368–371. IEEE
96. Zhu G, Zhao H, Liu H, Sun H (2019) A novel lstm-gan algorithm for time series anomaly detection. In: 2019 prognostics and system health management conference (PHM-Qingdao), pp. 1–6. IEEE
97. Shi J, Ding Y, Lv Z (2021) An intermittent fault data generation method based on lstm and gan. In: 2021 global reliability and prognostics and health management (PHM-Nanjing), pp. 1–4. IEEE
98. Jiang W, Hong Y, Zhou B, He X, Cheng C (2019) A gan-based anomaly detection approach for imbalanced industrial time series. IEEE Access 7:143608–143619
99. Huang T, Chakraborty P, Sharma A (2021) Deep convolutional generative adversarial networks for traffic data imputation encoding time series as images. Int J Transp Sci Technol. https://doi.org/10.1016/j.ijtst.2021.10.007
100. Hochreiter S (1998) The vanishing gradient problem during learning recurrent neural nets and problem solutions. Intern. J Uncertain Fuzziness Knowl-Based Syst 6(02):107–116
101. Bengio Y, Simard P, Frasconi P (1994) Learning long-term dependencies with gradient descent is difficult. IEEE Trans Neural Netw 5(2):157–166
102. Olszewski RT (2001) Generalized feature extraction for structural pattern recognition in time -series data. PhD thesis
103. Andrzejak RG, Lehnertz K, Mormann F, Rieke C, David P, Elger CE (2001) Indications of nonlinear deterministic and finite-dimensional structures in time series of brain electrical activity: dependence on recording region and brain state. Phys Rev E 64(6):061907
104. Akcay S, Atapour-Abarghouei A, Breckon TP (2018) Ganomaly: semi-supervised anomaly detection via adversarial training. In: Asian conference on computer vision, pp. 622–637. Springer
105. LeCun Y, Bottou L, Bengio Y, Haffner P (1998) Gradient-based learning applied to document recognition. Proc IEEE 86(11):2278–2324
106. Ramponi G, Protopapas P, Brambilla M, Janssen R (2018) T-cgan: Conditional generative adversarial network for data augmentation in noisy time series with irregular sampling. arXiv preprint arXiv:1811.08295
107. Mirza M, Osindero S (2014) Conditional generative adversarial nets. arXiv preprint arXiv:1411.1784
108. Chen G, Zhu Y, Hong Z, Yang Z (2019) Emotionalgan: generating ecg to enhance emotion state classification. In: Proceedings of the 2019 International conference on artificial intelligence and computer science, pp. 309–313
109. Sabir R, Rosato D, Hartmann S, Gühmann C (2021) Signal generation using 1d deep convolutional generative adversarial networks for fault diagnosis of electrical machines. In: 2020 25th international conference on pattern recognition (ICPR), pp. 3907–3914. IEEE
110. Kramer MA (1991) Nonlinear principal component analysis using autoassociative neural networks. AIChE J 37(2):233–243
111. Ni H, Szpruch L, Wiese M, Liao S, Xiao B (2020) Conditional sig-wasserstein gans for time series generation. arXiv preprint arXiv:2006.05421
112. Sakoe H, Chiba S (1978) Dynamic programming algorithm optimization for spoken word recognition. IEEE Trans Acoust Speech Signal Process 26(1):43–49
113. Forestier G, Petitjean F, Dau HA, Webb GI, Keogh E (2017) Generating synthetic time series to augment sparse datasets. In: 2017 IEEE International conference on data mining (ICDM), pp. 865–870. IEEE
114. Kamycki K, Kapuscinski T, Oszust M (2019) Data augmentation with suboptimal warping for time-series classification. Sensors 20(1):98
115. Shao J, Hu K, Wang C, Xue X, Raj B (2020) Is normalization indispensable for training deep neural network? Adv Neural Inf Process Syst 33:13434–13444
116. Sola J, Sevilla J (1997) Importance of input data normalization for the application of neural networks to complex industrial problems. IEEE Trans Nucl Sci 44(3):1464–1468
117. Zhang K (2021) On mode collapse in generative adversarial networks. In: International conference on artificial neural networks, pp. 563–574. Springer
118. Adiga S, Attia MA, Chang W-T, Tandon R (2018) On the tradeoff between mode collapse and sample quality in generative adversarial networks. In: 2018 IEEE global conference on signal and information processing (GlobalSIP), pp. 1184–1188. IEEE
119. Goodfellow I, Pouget-Abadie J, Mirza M, Xu B, Warde-Farley D, Ozair S, Courville A, Bengio Y (2020) Generative adversarial networks. Commun ACM 63(11):139–144
120. Barnett SA (2018) Convergence problems with generative adversarial networks (gans). arXiv preprint arXiv:1806.11382
121. Arjovsky M, Bottou L (2017) Towards principled methods for training generative adversarial networks. arXiv preprint arXiv:1701.04862
122. Gonog L, Zhou Y (2019) A review: generative adversarial networks. In: 2019 14th IEEE conference on industrial electronics and applications (ICIEA), pp. 505–510. IEEE
123. Lee O-Y, Shin Y-H, Kim J-O (2019) Multi-perspective discriminators-based generative adversarial network for image super resolution. IEEE Access 7:136496–136510
124. Wang X, Wang K, Lian S (2020) A survey on face data augmentation for the training of deep neural networks. Neural Comput Appl 32(19):15503–15531
125. Nalepa J, Marcinkiewicz M, Kawulok M (2019) Data augmentation for brain-tumor segmentation: a review. Front Comput Neurosci 13:83
126. Fu B, Kirchbuchner F, Kuijper A (2020) Data augmentation for time series: Traditional vs generative models on capacitive proximity time series. In: Proceedings of the 13th ACM international conference on pervasive technologies related to assistive environments. PETRA '20. Association for computing machinery, New York, NY (2020). https://doi.org/10.1145/3389189.3392606
127. Iglesias G, Talavera E, Díaz-Álvarez A (2022) A survey on GANs for computer vision: recent research, analysis and taxonomy. arXiv. https://doi.org/10.48550/ARXIV.2203.11242. https://arxiv.org/abs/2203.11242

128. González-Prieto Á, Mozo A, Gómez-Canaval S, Talavera E (2022) Improving the quality of generative models through smirnov transformation. Inf Sci 609:1539–1566

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.